

AGILE OS

Product Requirements Document

The Operating System for Engineering Teams

Version 1.0 — Draft
Status: Validation Phase
Date: March 2026

1. Executive Summary

Agile OS is a planning and intelligence platform that helps engineering teams stop repeating the same sprint failures. It connects five tightly integrated modules — Scope Cop, Sprint Brain, Dependency Radar, Velocity Mirror, and Retrospective AI — into one closed-loop system that learns from a team's history and improves with every sprint.

The core insight behind Agile OS is that sprint failure is rarely a single-point problem. Teams fail predictably because bad tickets enter planning unchecked, because capacity is estimated optimistically rather than historically, because dependency risks are known but ignored, because mid-sprint warning signs go unaddressed, and because retrospective learnings are never systematically acted on. Each of these problems has been addressed in isolation by existing tools. None have been connected into a single learning system.

Agile OS is that system. The longer a team uses it, the smarter it becomes. Historical data compounds into predictive intelligence. Each module feeds the next. The result is a team that becomes measurably more predictable, more reliable, and more efficient with every sprint cycle.

Key Value Proposition

- Eliminates sprint spillover by connecting planning, execution, and learning into one system
- Gets smarter the longer a team uses it — institutional memory becomes a competitive advantage
- Works on top of existing tools (Jira, Linear, GitHub Projects) — no migration required
- Replaces five disconnected point solutions with one unified platform

Metric	Target	Source
Sprint spillover reduction	50%+ within 3 months	Manual MVP validation
Planning time saved	60% reduction	Industry benchmarks
Sprint disruptions	30% fewer	Dependency Radar impact
Estimation accuracy	85%+ within 6 months	Historical velocity tracking
Team retention of learnings	3x improvement	Retrospective AI tracking

2. Problem Statement

2.1 The Core Problem

Engineering teams running agile sprints consistently fail to deliver what they commit to. Studies suggest teams lose as much as 10% of sprint capacity to administrative overhead alone. Sprint spillover — work that carries over unfinished into the next sprint — is endemic, demoralising, and economically costly.

The problem is not that teams lack tools. Jira, Linear, GitHub Projects, ClickUp, and dozens of others exist. The problem is that these tools record what happened but do not learn from it. They provide no forward-looking intelligence. They do not connect planning to execution to retrospective in a way that generates improvement over time.

2.2 Validated Pain Points

The following pain points were validated through direct practitioner research across multiple engineering teams and communities:

2.2.1 Bad Estimation

Developers consistently underestimate task complexity. This is not a motivation or competence problem — it is a data problem. Without historical per-developer velocity data by ticket type, estimates are based on optimism rather than evidence. Senior estimates are trusted; junior estimates are not calibrated against reality. Product Owners report flying blind on timelines whenever a senior developer is not personally handling the work.

2.2.2 Dependency Blindness

Teams regularly build sprints around external dependencies — suppliers, internal teams, stakeholders — whose reliability is known to be poor. The failure is not that the risk is invisible. It is that without a formal reliability record, optimism wins the room in planning. A supplier who has delivered on time twice in six sprints will be planned around as if they are reliable, because nobody has the data to argue otherwise. When they miss again, the team scrambles to replan, morale drops, and the sprint is ruined.

2.2.3 Invisible Mid-Sprint Failure

Sprint failures are often visible days before they become inevitable, but teams lack the monitoring infrastructure to catch them early enough to intervene. Queue time in code reviews, delayed acceptance criteria clarification, regression failures — these accumulate silently until the sprint is unsalvageable.

2.2.4 Retrospectives That Don't Stick

Most engineering teams run retrospectives that are feelings-based, inconsistently structured, and disconnected from data. Action items are captured but rarely tracked. The same problems recur sprint after sprint. Teams discuss the same issues in retro for months without systemic improvement because there is no mechanism to close the loop between identified problem and verified fix.

2.2.5 Ticket Quality

Vague, oversized, or poorly scoped tickets are the upstream cause of a large proportion of sprint failures. Stories that look straightforward at a high level routinely reveal significant hidden complexity once decomposed. Teams that become disciplined about decomposition and estimation — breaking work into small, independently shippable increments — consistently see spillover drop from near 50% of sprint capacity to under 10%. The problem is that this discipline currently depends on experienced practitioners who remember to push for it, rather than being systematically enforced.

2.3 What Existing Tools Miss

Tool Category	What It Does	What It Misses
Jira / Linear / GitHub Projects	Tracks tickets and sprint state	No forward-looking intelligence, no learning loop
Velocity trackers (Nave, etc.)	Visualises historical flow metrics	Reactive only — shows what happened, not what to do next
AI ticket assistants (Spinach, etc.)	Generates user stories and meeting notes	No connection to planning, execution, or retrospective
Capacity planners (ClickUp AI)	Estimates team load	No per-developer calibration, no dependency awareness
Retro tools	Structures retrospective meetings	No data integration, no action item tracking across sprints

3. Vision & Goals

3.1 Product Vision

Agile OS becomes the intelligence layer that sits across every stage of the sprint lifecycle — before planning, during planning, during execution, and after each sprint — continuously learning from a team's patterns and making the next sprint smarter than the last.

The vision is not to replace the tools teams already use. It is to make those tools dramatically smarter by connecting them into a closed learning loop that generates compound improvement over time.

3.2 Strategic Goals

- **Goal 1:** Reduce sprint spillover by 50% within three months of adoption for any engineering team.
- **Goal 2:** Create compounding institutional value — a team's Agile OS becomes more valuable the longer they use it, making switching costly.
- **Goal 3:** Become the standard planning intelligence layer for engineering teams of 5–50 developers at growth-stage companies.
- **Goal 4:** Generate \$100k ARR within 12 months of public launch through a bottom-up, product-led growth motion.

3.3 Non-Goals

The following are explicitly out of scope for the initial product:

- Replacing Jira, Linear, or GitHub Projects as the system of record for tickets
- Providing HR or performance management functionality
- Supporting non-engineering team workflows (initially)
- Real-time video or communication features
- Providing legal, compliance, or financial workflow management

4. Target Users

4.1 Primary Personas

Persona 1: The Pragmatic Scrum Master

Attribute	Description
Role	Scrum Master at a 20–100 person software company
Experience	2–5 years running agile teams
Core frustration	Spillover happens every sprint and retros don't fix it
Relationship to tools	Wants outcomes, not ideology — will use AI if it works
Buying power	Influences tool adoption, often holds a small software budget
Success metric	Sprints that finish what they start

Persona 2: The Engineering Manager

Attribute	Description
Role	Engineering Manager overseeing 2–4 teams
Experience	Former developer, now accountable for team output
Core frustration	Can't reliably forecast delivery dates to stakeholders
Relationship to tools	Cares about ROI and measurable improvement
Buying power	Direct budget authority for team tooling
Success metric	Predictable velocity and stakeholder trust

Persona 3: The Product Owner Under Pressure

Attribute	Description
Role	Product Owner managing a dev team of 4–12 developers
Experience	Deeply frustrated with chronic overcommitment
Core frustration	Guessing on timelines; junior vs senior performance gap is unmanaged
Relationship to tools	Wants data to have honest planning conversations with the team
Buying power	Recommends tools; needs engineering manager sign-off
Success metric	Commitments the team actually meets

4.2 Secondary Markets

Research has identified a secondary market beyond software development teams. Engineering teams in hardware, manufacturing, R&D, and construction running agile or agile-adjacent processes experience identical spillover problems. This market is underserved by software-specific tools and represents a significant expansion opportunity after initial product-market fit is established.

5. Product Overview

Agile OS is a five-module platform connected by a continuous learning loop. Each module addresses one stage of the sprint lifecycle. Together, they form a closed system where the output of each stage becomes the input intelligence for the next.

The Agile OS Loop

Ticket Created → Scope Cop reviews it
 Sprint Planning → Sprint Brain + Dependency Radar build the plan
 Sprint Execution → Velocity Mirror monitors live
 Sprint End → Retrospective AI generates learnings
 Next Sprint → System is smarter than before

The platform integrates with Jira, Linear, and GitHub Projects as read/write data sources. It does not replace these tools — it sits on top of them as an intelligence layer, surfacing insights and recommendations at the moment they are most useful.

6. Module Specifications

6.1 Scope Cop — Ticket Quality Enforcer

Phase: Before Sprint Planning

Problem Solved: Vague, oversized, or poorly estimated tickets that cause sprint failure before it starts

6.1.1 Core Functionality

- Analyses every ticket at creation time against the team's historical ticket database
- Flags tickets where the written estimate diverges significantly from historical completion time for similar work
- Identifies tickets with insufficient acceptance criteria based on historical patterns of missed criteria
- Detects scope ambiguity by comparing ticket descriptions to tickets that historically required clarification mid-sprint
- Suggests ticket splits for stories estimated above 8 story points, generating specific split recommendations based on similar past decompositions
- Monitors ticket description edits throughout the sprint for scope expansion and alerts the PM when expansion exceeds a configurable threshold

6.1.2 User Interactions

- Inline ticket review widget in Jira/Linear that surfaces Scope Cop analysis alongside ticket creation
- Sprint backlog health score showing aggregate ticket quality before planning begins
- One-click acceptance criteria template generation based on ticket type and historical patterns
- Developer feedback loop: post-sprint, developers rate ticket quality; this feeds back into the estimation model

6.1.3 Success Metrics

- Reduction in tickets requiring mid-sprint scope clarification
- Reduction in tickets that exceed their original estimate by more than 50%
- Developer satisfaction with ticket quality at sprint start

6.2 Sprint Brain — AI Sprint Planner

Phase: Sprint Planning

Problem Solved: Overcommitment due to optimistic estimation and failure to account for per-developer capacity variance

6.2.1 Core Functionality

- Builds per-developer velocity profiles by ticket type, complexity, and domain from historical sprint data
- Generates sprint plans that match tickets to developers based on actual historical delivery patterns, not generic story point estimates
- Factors in PTO, known meetings, context-switching costs, and part-time capacity automatically

- Targets 90–95% capacity utilisation per developer, leaving buffer for unexpected blockers
- Provides natural-language explanations for every assignment recommendation: 'Assigning this authentication ticket to Sarah based on her completion of 4 similar tickets in the last 3 sprints at an average of 2.1 days'
- Surfaces historical data as evidence in planning conversations, giving developers objective grounds to push back on overcommitment

6.2.2 The Junior/Senior Calibration Problem

A critical use case identified in user research is the gap between junior and senior developer estimation reliability. Sprint Brain addresses this by maintaining independent velocity profiles per developer rather than team averages. A junior developer's estimate on an API integration ticket is automatically weighted against how long that developer's API integration tickets have historically taken, not against a team average inflated by senior performance. This gives Product Owners and Engineering Managers objective data to calibrate expectations without it feeling like surveillance or punishment.

6.2.3 User Interactions

- Sprint planning wizard that walks the team through AI-generated sprint composition with reasoning
- What-if scenario modeller: drag tickets in and out of the sprint and see projected completion probability update in real time
- Capacity override controls: mark individual developers as reduced capacity for a sprint with reasons
- Sprint commitment confidence score: percentage probability of completing all committed work based on historical patterns

6.2.4 Success Metrics

- Reduction in sprint spillover rate
- Improvement in sprint commitment accuracy (committed vs delivered story points ratio)
- Reduction in time spent in sprint planning meetings

6.3 Dependency Radar — Dependency & Supplier Tracker

Phase: Sprint Planning & Pre-Sprint

Problem Solved: Teams building sprints around unreliable external dependencies because reliability history is invisible

6.3.1 Core Functionality

- Maintains a reliability score for every dependency — internal teams, external suppliers, stakeholders — based on their actual delivery history
- At sprint planning, automatically flags any ticket whose completion depends on a low-reliability dependency
- Surfaces the specific reliability record: 'This supplier has delivered on time 2 of the last 6 sprints. Your sprint plan has 40% of capacity blocked on their delivery.'
- Sends automated nudges to dependency owners when their work is on the critical path for an upcoming sprint
- Provides two sprint plan variants: one assuming dependencies deliver, one contingency plan if they do not
- Tracks and scores internal team dependencies with the same reliability framework as external ones

6.3.2 The Core Insight

Research validated that the most common cause of dependency-related sprint failure is not that the risk was unknown — it is that the risk was known but there was no data to override the planning optimism. Dependency Radar solves this by making the reliability record formal, visible, and present in the room at planning time. Teams can still choose to plan optimistically — but they do so with the evidence in front of them, not hidden.

6.3.3 User Interactions

- Dependency map view showing all sprint tickets and their external dependencies
- Reliability scorecards for each dependency source, updated automatically after each sprint
- Pre-sprint risk report: 'Your sprint has 3 high-risk dependencies. Here is your contingency plan if any of them slip.'
- Automated stakeholder notifications with visibility into how their delivery affects the sprint

6.3.4 Success Metrics

- Reduction in sprints ruined by dependency failure
- Improvement in dependency owner on-time delivery rates (Hawthorne effect of visibility)
- Reduction in mid-sprint replan events

6.4 Velocity Mirror — Live Sprint Intelligence Dashboard

Phase: During Sprint Execution

Problem Solved: Sprint failure that is visible days before it becomes inevitable but goes unaddressed because there is no monitoring infrastructure

6.4.1 Core Functionality

- Real-time dashboard tracking sprint progress against the AI-generated burndown path from Sprint Brain
- Per-developer capacity utilisation monitoring with automatic alerts when someone is trending over or under capacity
- Queue time detection: identifies when tickets are stalled in review, waiting for clarification, or blocked by dependencies
- Sprint health score updated in real time, with colour-coded status and specific actionable recommendations
- Predictive failure alerts: 'At current velocity, 3 tickets are at risk of spillover. Recommended action: descope ticket #247 or reassign to available developer.'
- Root cause tagging: when tickets slip, automatically tags the cause category (estimation, dependency, scope creep, quality) to build pattern data for retrospectives

6.4.2 User Interactions

- Live sprint dashboard accessible to all team members — full transparency by default
- PM and Scrum Master alert feed with recommended interventions ranked by impact
- Developer-facing view showing individual workload and upcoming capacity
- One-click escalation: surface a dependency block directly to the dependency owner from the dashboard

6.4.3 Success Metrics

- Reduction in sprint failures that were not caught until the final days
- Improvement in mid-sprint intervention rate when alerts are triggered
- Reduction in queue time per ticket across the sprint

6.5 Retrospective AI — Automated Sprint Learning Engine

Phase: After Sprint / Continuous

Problem Solved: Retrospectives that are feelings-based, produce action items that are never tracked, and fail to generate systemic improvement

6.5.1 Core Functionality

- Automatically generates a full data-driven retrospective report after every sprint closes — no manual effort required
- Report includes: what was committed, what was delivered, spillover analysis, root cause breakdown by category, velocity trend, and dependency performance
- Identifies patterns across multiple sprints: 'Authentication tickets have exceeded their estimate in 5 of the last 7 sprints. This represents a systematic estimation problem, not individual failure.'
- Extracts action items from the retrospective and creates trackable commitments in the team's project management tool
- At the start of each subsequent sprint, surfaces outstanding action items from previous retrospectives and reports on whether they were implemented
- Generates a Team Health Score — a rolling metric combining sprint predictability, velocity consistency, improvement trend, and retrospective action item follow-through rate

6.5.2 The Institutional Memory Problem

Most engineering teams have no systematic memory. The same problems recur because there is no mechanism that connects 'we identified this problem in sprint 14' to 'we are experiencing this problem again in sprint 22.' Retrospective AI is that mechanism. It holds the team accountable to its own commitments across time, and surfaces patterns that would be invisible in any single retrospective.

6.5.3 User Interactions

- Auto-generated retro report delivered to the team at sprint close — viewable in Agile OS and exportable to Confluence or Notion
- Action item tracker with owner assignment, due date, and sprint-over-sprint completion tracking
- Team Health Score dashboard with trend over time and benchmark comparison to anonymised similar teams
- Retro meeting facilitator mode: use the AI report as the agenda for a live retrospective, with discussion prompts generated from the data

6.5.4 Success Metrics

- Increase in retrospective action item completion rate
- Reduction in recurring problem categories across sprints
- Improvement in Team Health Score over time

7. Technical Requirements

7.1 Integrations

Tool	Integration Type	Priority
Jira	OAuth read/write via REST API	P0 — Launch requirement
Linear	OAuth read/write via GraphQL API	P1 — Month 3
GitHub Projects	OAuth read via REST API	P1 — Month 3
Confluence	Write — retro report export	P2 — Month 6
Notion	Write — retro report export	P2 — Month 6
Slack	Alerts, nudges, notifications	P1 — Month 3
Google Calendar	Capacity planning — PTO and meeting load	P2 — Month 6

7.2 AI & Machine Learning Requirements

- Per-developer velocity models must be trained on a minimum of 3 completed sprints before Sprint Brain recommendations are surfaced
- Estimation accuracy models must improve measurably with each additional sprint of data
- All AI recommendations must include confidence scores and the data basis for the recommendation
- Models must be team-specific — no cross-team data sharing without explicit opt-in
- LLM-based features (ticket analysis, retro generation) must use structured prompts with output validation

7.3 Data & Privacy

- All sprint data is stored per-organisation with strict tenant isolation
- No team data is used to train models for other teams without explicit consent
- GDPR and SOC 2 Type II compliance required before enterprise sales motion
- Data export available at any time in standard formats (CSV, JSON)
- Data deletion within 30 days of account closure

7.4 Performance Requirements

Feature	Performance Requirement
Sprint plan generation	< 30 seconds for teams up to 20 developers
Live dashboard refresh	< 5 second latency from Jira/Linear update
Retro report generation	< 2 minutes after sprint close

Ticket analysis (Scope Cop)	< 10 seconds per ticket
API uptime	99.9% SLA

8. Go-to-Market Strategy

8.1 Phased Launch Plan

Phase	Timeline	Focus	Revenue Target
Validation	Now — Month 1	Manual MVPs with 5 teams, zero code	£0 — learning only
Sprint Brain MVP	Month 1–3	Jira integration, velocity profiles, beta teams	\$99/team/month — 10 teams
Velocity Mirror	Month 4–6	Live dashboard, alerts, repricing	\$199/team/month — 25 teams
Scope Cop + Dependency Radar	Month 7–9	Full planning suite, expand integrations	\$299/team/month — 50 teams
Retrospective AI + Full Platform	Month 10–12	Enterprise packaging, SSO, admin controls	\$2k–5k/month — enterprise

8.2 Positioning

Agile OS is not positioned as an 'AI tool.' Research validated that this framing actively repels the target audience — practitioners exhausted by AI hype who have been burned by tools that overpromise and underdeliver. Agile OS is positioned around the outcome:

Positioning Statement

"The planning and intelligence platform that helps engineering teams stop repeating the same sprint failures."

We never lead with AI. We lead with: predictable sprints, reliable forecasts, and teams that actually improve over time.

8.3 Pricing Model

Tier	Price	Included	Target
Starter	\$99/team/month	Sprint Brain + Velocity Mirror, up to 10 developers	Small teams, validation phase
Growth	\$299/team/month	All 5 modules, up to 25 developers	Mid-size teams, primary market
Enterprise	Custom (\$2k–5k+/month)	All modules, SSO, admin controls, SLA, dedicated support	50+ developer orgs

8.4 Acquisition Channels

- Product-led growth via free trial of Sprint Brain with Jira integration — value demonstrated before payment required
- Community-led growth via engineering manager and Scrum Master communities (Rands Leadership Slack, r/agile, r/scrum, Agile Alliance)
- Content marketing targeting 'sprint spillover', 'sprint planning', and 'engineering team productivity' search intent
- Bottom-up enterprise motion: individual team adoption within a larger org creates internal advocates who drive company-wide rollout

9. Success Metrics & KPIs

9.1 Product Metrics

Metric	Target	Measurement Method
Sprint spillover rate (user teams)	< 15% within 3 months	Jira/Linear data analysis
Estimation accuracy improvement	> 85% within 6 months	Committed vs delivered story points
Dependency-related sprint failures	50% reduction	Root cause tagging in Velocity Mirror
Retro action item completion	> 60% per sprint	Retrospective AI tracking
Team Health Score improvement	Positive trend for 80% of teams	Rolling platform metric
Daily active usage (per team)	> 3 team members per sprint week	Session analytics

9.2 Business Metrics

Metric	Month 6 Target	Month 12 Target
Paying teams	25	100+
ARR	\$50k	\$100k+
Monthly churn	< 5%	< 3%
NPS	> 40	> 60
CAC payback period	< 6 months	< 4 months

10. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
AI fatigue — target users reject tool because of AI positioning	High	High	Lead with outcomes, never AI. Use language like 'sprint analytics' and 'planning intelligence.'
Agile purists argue tooling addresses symptoms not causes	High	Medium	Don't target purists. Target pragmatic practitioners who need wins now, not culture overhauls.
Jira API changes break core integration	Medium	High	Abstract integration layer; support Jira, Linear, and GitHub Projects from day one to reduce single-point dependency.
Insufficient data for ML models in early sprints	High	Medium	Surface manual insights for teams with fewer than 3 sprints of data; gate AI features behind data thresholds.
Competitor (Jira, ClickUp) copies core features	Medium	High	Moat is the cross-sprint learning loop and institutional memory — this takes months of data to replicate.
Teams resist per-developer tracking as surveillance	Medium	Medium	Frame as team-level intelligence, not individual performance management. Make individual data private by default.

11. Open Questions

The following questions require validation before final product decisions can be made:

- What is the minimum number of sprints of historical data required for Sprint Brain recommendations to be meaningfully accurate? (Hypothesis: 3 sprints minimum, 8 sprints for high confidence)
- Should per-developer velocity data be visible to the developer themselves, to their manager, or both? What are the trust and culture implications of each option?
- Is the primary buyer the Scrum Master, the Engineering Manager, or the CTO? Research suggests different personas in different org sizes — how does this affect the sales motion?
- Is there sufficient demand from non-software engineering teams (hardware, manufacturing, R&D) to pursue this as a secondary market in year one, or does it dilute focus?
- What is the correct pricing model for early-stage teams with limited historical data who cannot yet access the full value of the AI features?
- Should Dependency Radar include automated stakeholder nudges by default, or make this opt-in to avoid backlash from suppliers and internal teams who feel monitored?

12. Appendix — Validation Research Summary

12.1 Research Methodology

Primary research was conducted via posts in r/agile and r/scrum communities, generating 40+ practitioner responses. Respondents included Scrum Masters, Engineering Managers, Product Owners, senior developers, and Agile Coaches. The research was designed to validate or invalidate the core hypothesis before any product development began.

12.2 Key Findings

- Sprint spillover is universally acknowledged as a real and costly problem across all team types and sizes
- The root cause varies significantly: estimation failure (most common), dependency blindness (very common), cultural dysfunction (common in mature teams), and technical debt (common in legacy codebases)
- 'AI tool' framing is actively counterproductive with the target audience — outcome-led positioning is essential
- The strongest pain signal came from a Product Owner whose team experienced spillover every sprint with no improvement despite trying multiple interventions
- A non-software engineering team reported reducing spillover from ~50% of sprint capacity to under 10% through manual discipline — validating the potential impact of systematising that discipline
- Dependency management emerged as an underserved and highly painful problem not addressed by any existing tool

- Experienced practitioners (Agile purists) are not the target market — they have solved the problem through culture change and are philosophically opposed to tooling solutions

12.3 Competitive Intelligence

The following tools were identified as operating in adjacent spaces:

- Nave — Flow metrics and cycle time visualisation. Reactive, not predictive. No planning intelligence.
- Kanban+ Flowmetrics — Similar to Nave. Strong on historical analysis, weak on forward-looking recommendations.
- Spinach.io — AI Scrum Master focused on meeting notes and ticket updates. No cross-sprint learning loop.
- Jira Advanced Roadmaps — Enterprise capacity planning. Complex, expensive, not accessible to SMB teams.
- ClickUp AI — General-purpose AI features bolted onto a project management tool. Not sprint-specific.

No existing tool connects sprint planning, execution monitoring, and retrospective learning into a unified system with a continuous learning loop. This is the white space Agile OS occupies.

Agile OS — PRD v1.0

Document status: Living document. Update after each validation milestone.